GuardianDash

Smart Driving. Safer Lives.

Final Project — Embedded Systems Course

PRESENTED TO

Dr. Doaa Shoeib

Eng. Marwan · Eng. Rehab

Team Members

GuardianDash was designed, built and shipped by a four-person team across hardware, firmware, mobile and backend layers.

#	Name	Student ID
1	Rimas Emad	221001067
2	Mohamed Eldairouty	221001719
3	Moaz Ali	221001970
4	Judy Yehia	221000495

Project Overview

GuardianDash is a smart vehicle black-box that watches over the driver on every trip. A small STM32-based device inside the car continuously reads G-force from an MPU6050 accelerometer. The moment it detects a collision it shows STATUS: UNSAFE on its on-board LCD, streams the spike to a paired smartphone over Bluetooth, and automatically calls or messages the driver's emergency contacts — all in seconds.

The system is end-to-end and standalone: the hardware lives sealed inside the vehicle on a USB power bank, the mobile app runs as a real native Android APK, and a small cloud backend persists user accounts and trip history. No laptop is required during a real drive.

System Architecture

The product is split into three independent components. Each one can be developed, tested and deployed on its own, and they communicate through well-defined wireless interfaces.

Hardware — Vehicle_BlackBox

- STM32F401 microcontroller running custom firmware in C
- MPU6050 IMU over I²C, computing $G = \sqrt{(Ax^2 + Ay^2 + Az^2)}$
- 16×2 I²C LCD showing live G-force and SAFE / UNSAFE status
- HM-10 BLE module streaming telemetry to the phone over Bluetooth
- Powered by a USB power bank for full mobility

Mobile App — GuardianDash (Android APK)

- React Native + Expo + TypeScript codebase
- Live dashboard mirroring the on-board LCD in real time
- Full-screen crash alert with 30-second auto-call countdown
- Bluetooth pairing flow, emergency contacts CRUD, trip history
- Real GPS via expo-location, dark themed UI throughout

Cloud Backend — GuardianDash API

- Node.js + Express REST API with JWT authentication
- SQLite database storing users, contacts, trips and stats
- Hosted on Render.com (free tier) over HTTPS
- Auto-deploys from GitHub on every push

Key Features

Crash detection & emergency response

- Configurable G-force threshold (Low 2.0g · Medium 1.5g · High 1.2g)
- Full-screen red alert with 30-second cancel window
- Auto-call to the highest-priority emergency contact
- Parallel SMS draft with location and severity attached
- GPS coordinates included in the outgoing alert

Live telemetry

- Real-time G-force readout mirroring the hardware LCD
- Accelerometer X / Y / Z displayed alongside total G
- Trip history with map polyline, distance, max and average speed
- Cloud sync — same account works across multiple devices

Polished native UX

- Dark, modern UI built for clarity in any lighting
- Entrance animations and haptic feedback on every action
- Native Android APK — installs and runs without Expo Go

Tools & Technologies

The stack was chosen for productivity, maintainability and cost — every layer is free for educational and prototype use.

Layer	Tools / Components
Mobile App	React Native · Expo SDK 51 · TypeScript · Expo Router · Reanimated · Zustand
Maps	OpenStreetMap tiles · react-native-svg
Bluetooth	react-native-ble-plx (HM-10 BLE)
Backend	Node.js · Express · SQLite · JWT
Hosting	Render.com (free tier · auto-deploy from GitHub)
Hardware	STM32F401 · MPU6050 IMU · 16×2 I²C LCD · HM-10 BLE · USB power bank
Firmware	C · STM32 HAL · I²C1 @ 100 kHz · USART2 @ 9600 baud
Dev Tools	STM32CubeIDE · EAS Build · Git / GitHub

Closing

GuardianDash bridges three disciplines we explored throughout the Embedded Systems course: low-level sensor acquisition on the STM32, wireless data transport via BLE, and end-user-facing mobile and cloud development. The result is a fully working prototype that turns an accelerometer reading into a real-world emergency response in under a second.

GuardianDash — Smart Driving. Safer Lives.